

Relazione di Progetto di Basi di Dati



Deano Luca 159357
Mancardi Devin 159108
Mauro Gianfranco 157548
Volpi Davide 157048

A.S. 2023/24 - IBML

Contents

1	Sezione Gestionale	3
1.1	Definizione delle macro-fasi	3
1.2	Suddivisione dei compiti	3
2	Raccolta e analisi dei requisiti	4
2.1	Raccolta dei requisiti	4
2.2	Analisi dei requisiti	4
2.2.1	Riscrittura dei requisiti	4
2.2.2	Operazioni	6
2.2.3	Considerazioni sulle operazioni	6
3	Progettazione cconcettuale	7
3.1	Progettazione dello schema	7
3.1.1	Entità	7
3.1.2	Relazioni	9
3.2	Schema ER	11
4	Progettazione Logica	12
4.1	Preparazione dello schema per la conversione	12
4.1.1	Analisi delle ridondanze	12
4.1.2	Rimozione delle generalizzazioni	15
4.1.3	Rimozione degli attributi composti	15
4.1.4	Definizione delle chiavi primarie	15
4.1.5	Schema ER ristrutturato	16
4.2	Conversione dello schema	17
4.2.1	Vincoli di integrità	17
4.3	Schema Logico	18
5	Progettazione Fisica	19
5.1	Scelta dei tipi	19
5.2	Schema Fisico	21
6	Implementazione	22
6.1	Creazione del database e delle tabelle	22
6.2	Trigger	25
6.3	Implementazione delle operazioni	26
7	Analisi dei dati in R	28
7.1	Popolamento della base di dati	28
7.2	Analisi grafica delle operazioni	30
7.2.1	Considerazioni grafiche	34
7.3	Stima della quantità di memoria	35

1 Sezione Gestionale

1.1 Definizione delle macro-fasi

Per la progettazione è richiesto che vengano seguite le seguenti fasi:

- Raccolta e analisi dei requisiti
- Progettazione concettuale
- Progettazione logica
- Progettazione fisica
- Implementazione
- Analisi dei dati in R

A queste andiamo ad aggiungere il compito di stesura della relazione.

1.2 Suddivisione dei compiti

Compito	Sottocompiti	Svolti da
Stesura relazione		Volpi Mauro
Analisi dei requisiti		Mauro Mancardi
Progettazione concettuale	Riscrittura dei requisiti Creazione dello schema ER Valutazione dello schema	Volpi Mauro Mancardi Deano
Progettazione logica	Ristrutturazione logica Creazione schema logico	Volpi Mauro
Progettazione Fisica		Deano Mancardi
Implementazione		Volpi Mauro
Analisi in R		Volpi Mancardi
Stima della quantità di memoria		Volpi Deano

Nonostante la tabella indichi le responsabilità principali, tutti i componenti del gruppo hanno lavorato in tutte le fasi per la revisione e il controllo.

2 Raccolta e analisi dei requisiti

2.1 Raccolta dei requisiti

In questa fase viene stabilito cosa la base di dati dovrà fare. Vengono specificate le funzionalità e le qualità che deve possedere, senza vincolare la progettazione e l'implementazione.

Si vuole progettare una base di dati contenente informazioni sui principali laghi e fiumi europei. Le informazioni più importanti di cui tener conto sono le seguenti:

Si vuole tener traccia delle diverse nazioni europee. Di ogni nazione, identificata univocamente dal proprio nome, vengono registrate le seguenti informazioni: popolazione (relativa al mese di gennaio 2016), superficie, capitale e nazioni europee confinanti.

Di ogni fiume vengono registrati il nome, che lo identifica univocamente, le nazioni europee attraversate, per ogni nazione europea attraversata, la lunghezza del relativo tratto di fiume e la lunghezza totale del fiume. Limitatamente ai fiumi registrati nella base di dati, si vuole, inoltre, rappresentare il fatto che un fiume possa essere affluente di un altro fiume.

Di ciascun lago, identificato univocamente dal nome, interessano la superficie, la profondità massima e le nazioni europee da esso bagnate. Limitatamente ai fiumi registrati nella base di dati, devono essere rappresentati anche gli eventuali immissari e emissari di ogni lago.

2.2 Analisi dei requisiti

2.2.1 Riscrittura dei requisiti

Si vuole progettare una base di dati contenente informazioni sui principali laghi e fiumi europei. Le informazioni più importanti di cui tener conto sono le seguenti:

Ogni FIUME:

- È identificato univocamente dal nome
- Ha un attributo derivato, la lunghezza totale
- Ha due generalizzazioni:
 - EMISSARIO
 - Può essere emissario di un lago
 - IMMISSARIO
 - Può essere immissario di un lago
- Può sfociare in un solo mare

- Può essere affluente di un solo altro fiume
- Può avere degli affluenti
- Attraversa una o più nazioni per un determinato tratto

Ogni NAZIONE:

- È identificata univocamente dal nome
- Ha un attributo popolazione, superficie e capitale
- Può confinare con altre nazioni
- Può essere attraversata da dei fiumi
- Può essere bagnata da dei laghi per una determinata superficie

Ogni LAGO:

- È identificato univocamente dal nome
- Ha un attributo composto CARATTERISTICHE:
 - Profondità
 - Un attributo derivato, la superficie
- Bagna almeno una nazione per una determinata superficie
- Può avere dei fiumi come emissari
- Può avere dei fiumi come immissari

Ogni MARE:

- È identificato univocamente dal nome
- Possono sfociarci dei fiumi

2.2.2 Operazioni

Indichiamo un insieme di operazioni individuate per l'uso tipico del database. Queste operazioni verranno usate per l'analisi delle prestazioni durante la progettazione e nell'implementazione finale.

Id	Operazioni
1	Trovare tutte le nazioni bagnate da 3 o più laghi
2	Trovare tutte le nazioni bagnate da 1 o più laghi da cui esce un fiume che sfocia nel Mar Baltico
3	Trovare il numero di fiumi che sono immissari in un lago che ha al più 3 emissari
4	Trovare tutti i fiumi che sono sia immissari che emissari
5	Aggiornamento della superficie di un lago
6	Aggiornamento della lunghezza di un fiume
7	Calcolare la lunghezza totale di un fiume
8	Calcolare la superficie totale di un lago
9	Calcolare la distribuzione dei fiumi nelle nazioni
10	Calcolare la distribuzione dei laghi nelle nazioni

2.2.3 Considerazioni sulle operazioni

In questo sottocapitolo vengono fatte delle considerazioni sulle operazioni da implementare:

- Non vengono contemplate operazioni di inserimento, se non durante il popolamento, in quanto la base di dati non prevede la nascita di nuovi laghi o fiumi in Europa, i confini nazionali e le nazioni, come anche i mari sono definiti inizialmente.
- Non sono contemplate operazioni di cancellazione in quanto la base di dati nel caso di prosciugamento di un lago o un fiume procederà con l'aggiornamento della sua superficie/lunghezza. Analogamente anche le nazioni e/o i confini prevediamo che non subiranno variazioni geopolitiche.

3 Progettazione concettuale

3.1 Progettazione dello schema

Con tecnica mista andremo a individuare i concetti principali e a realizzare uno schema scheletro. Sulla base di questo verrà decomposto, raffinato, espanso e integrato fino ad arrivare allo schema finale.

3.1.1 Entità

In questo paragrafo troviamo la definizione delle entità. Abbiamo Fiume, Lago e Nazione come entità fondamentali. In aggiunta andiammo a definire una nuova entità Mare.

Entità Fiume

Dai requisiti otteniamo che è identificato univocamente dal nome e come attributo ha la sua lunghezza, che verrà derivata. Per poter discriminare i fiumi che sono emissari o immissari di altri fiumi usiamo per il momento una generalizzazione parziale con sovrapposizione, per garantire la flessibilità dello schema.

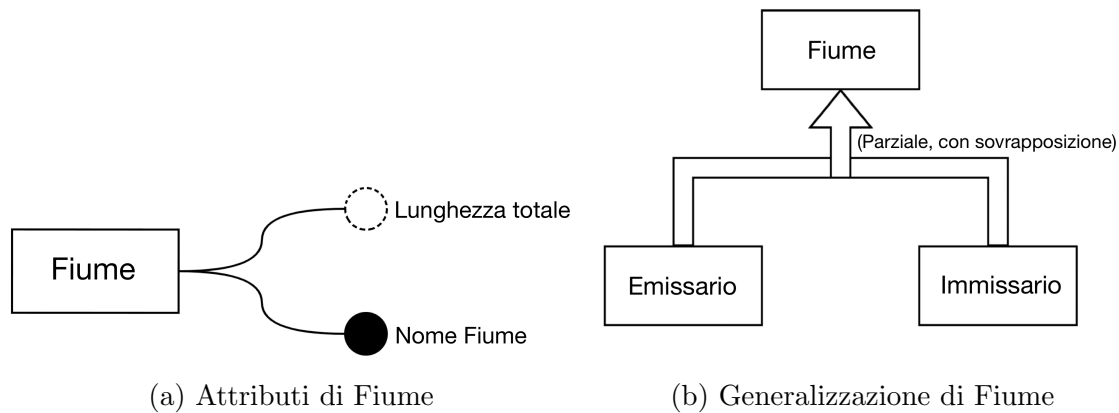


Figure 1: Entità Fiume

Entità Lago

Dai requisiti otteniamo che è identificato univocamente dal nome. Ha una serie di attributi: profondità e una superficie, che verrà derivata.

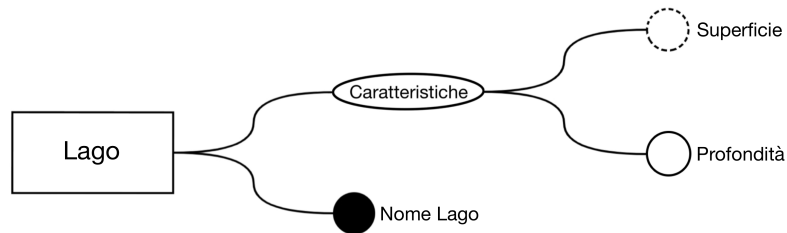


Figure 2: Entità Lago

Entità Nazione

Dai requisiti otteniamo che è identificata univocamente dal nome, ha un attributo popolazione.

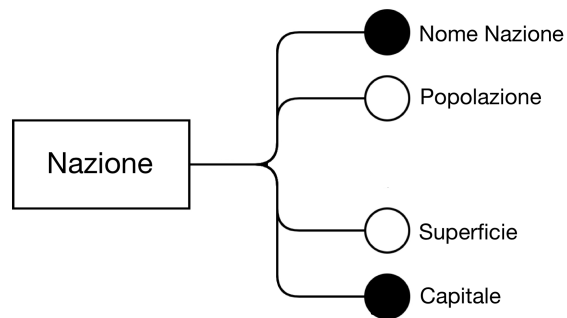


Figure 3: Entità Nazione

Entità Mare

Dai requisiti assumiamo che un fiume possa sfociare in un mare. Creiamo l'entità identificata univocamente dal nome.

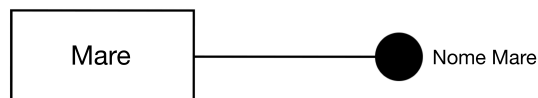


Figure 4: Entità Mare

3.1.2 Relazioni

In questo paragrafo troviamo la definizione delle relazioni. Individuiamo come principali le relazioni Attraversa, Confina, Bagna, Sfocia, E' affluente_di, Esce ed Entra.

Relazione Attraversa

Questa relazione lega Fiume e Nazione. Un fiume può attraversare una o più nazioni e ogni nazione può essere attraversata da più fiumi. L'attributo della relazione Lunghezza_tratto rappresenta la lunghezza del tratto del fiume nello stato a cui è legato dalla relazione.

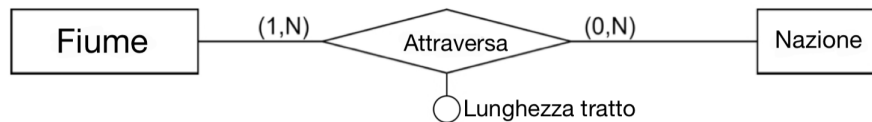


Figure 5: Relazione Attraversa

Relazione Confina

Questa relazione lega Nazione con Nazione in modo ricorsivo. Una nazione può confinare con più nazioni e una nazione può essere confine di più nazioni.

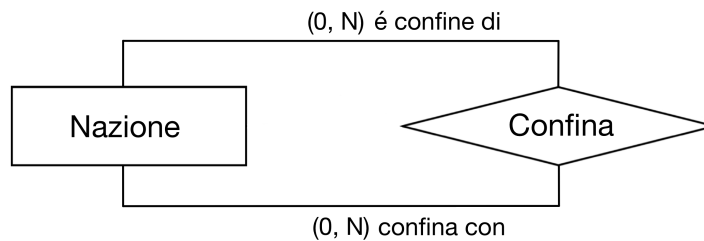


Figure 6: Relazione ricorsiva di Confina

Relazione Bagna

Questa relazione lega Lago con Nazione. Un lago può bagnare una o più nazioni e una nazione può non essere bagnata da più laghi. L'attributo della relazione Superficie_bagnata rappresenta la superficie del lago nello stato a cui è legato.

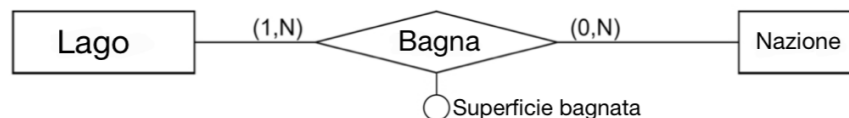


Figure 7: Relazione Bagna

Relazione Sfocia

Questa relazione lega Fiume con Mare. Un fiume può sfociare in al più un mare e in un mare possono sfociarci più fiumi.



Figure 8: Relazione Sfocia

Relazione E' affluente di

Questa relazione lega Fiume con Fiume in modo ricorsivo. Un fiume può essere affluente di al più un fiume e in un fiume possono affluire più fiumi.

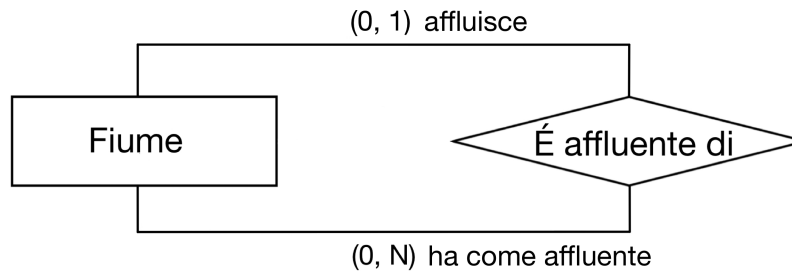


Figure 9: Relazione ricorsiva di Affluenza

Relazione di Immissione ed Emissione

Collegandoci alla generalizzazione di Fiume vista nel capitolo precedente otteniamo le due relazioni: Esce che lega Emissario con Lago e Entra che lega Immissario con Lago. Un immissario può sfociare in al più un lago e in un lago possono sfociarci più immissari. Un emissario può crearsi da un solo lago e da un lago possono esserci più emissari.

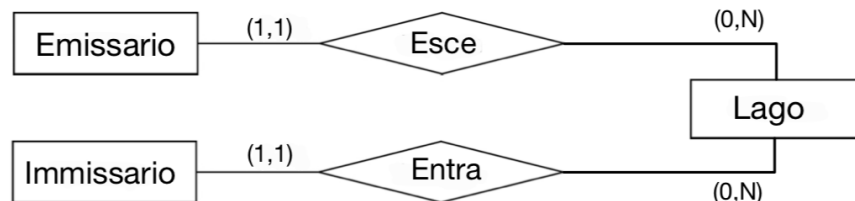


Figure 10: Relazione di immissione ed emissione

3.2 Schema ER

Unendo tutte le relazioni e le entità definite nella progettazione otteniamo il seguente schema entità relazioni:

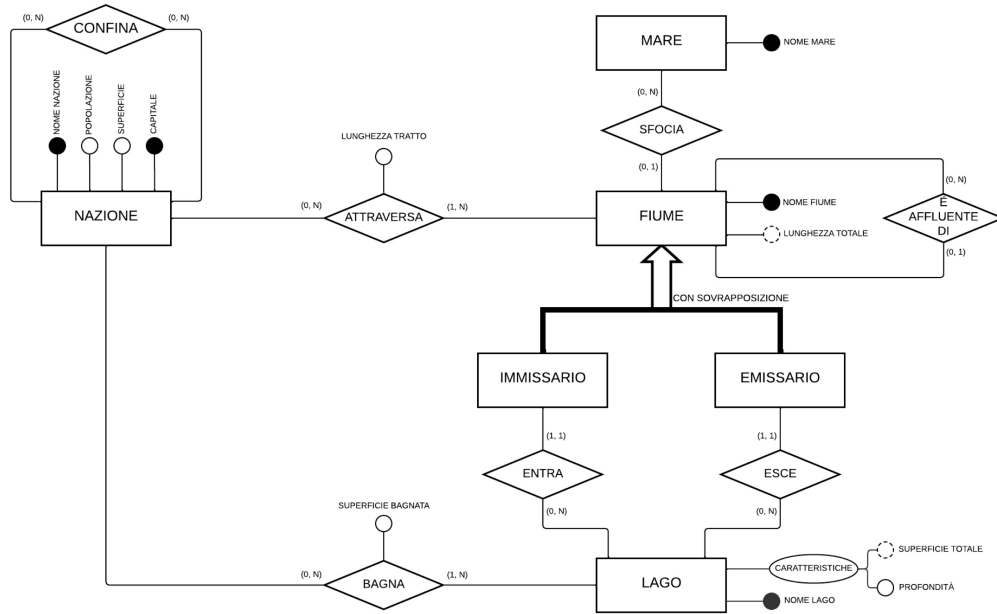


Figure 11: Schema ER completo

Regole di derivazione

Lunghezza totale:

$$\text{Dato un Fiume} \sum_{lunghezza_tratto} \forall Attraversa(Fiume, Nazione)$$

Superficie totale:

$$\text{Dato un Lago} \sum_{superficie_bagnata} \forall Bagna(Lago, Nazione)$$

4 Progettazione Logica

4.1 Preparazione dello schema per la conversione

4.1.1 Analisi delle ridondanze

In questa sezione analizziamo le ridondanze che coinvolgono gli attributi derivati individuati nello schema ER.

Volume operazioni

In questa sezione riportiamo la frequenza di ogni operazione definita, con tipo di operazione, se batch o iterativa:

Id	Operazioni	Frequenza	Tipo
1	Aggiornamento della superficie di un lago	30 all'anno	I
2	Aggiornamento della lunghezza di un fiume	20 all'anno	I
3	Calcolare la lunghezza totale di un fiume	30 all'anno	I
4	Calcolare la superficie totale di un lago	40 all'anno	I

Volume Entità e Relazioni

In questo punto definiamo i volumi delle entità e delle relazioni coinvolti nelle operazioni analizzate:

Entità/Relazione	Tipo	Volume
Fiume	Entità	100000
Immissario	Entità	30000
Emissario	Entità	30000
Lago	Entità	80000
Mare	Entità	9
Nazione	Entità	46
Confina	Relazione	69
Attraversa	Relazione	50000
E' Affluente Di	Relazione	60000
Entra	Relazione	2300
Esce	Relazione	2500
Bagna	Relazione	6000

Costo dell'operazione 1 con ridondanza:

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Lago	E	1	W
Nazione	E	1	R
Bagna	R	1	W

Costo dell'operazione 1 senza ridondanza:

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Lago	E	1	R
Nazione	E	1	R
Bagna	R	1	W

Costo dell'operazione 2 con ridondanza:

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Fiume	E	1	W
Nazione	E	1	R
Attraversa	R	1	W

Costo dell'operazione 2 senza ridondanza:

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Fiume	E	1	R
Nazione	E	1	R
Attraversa	R	1	W

Costo dell'operazione 3 con ridondanza:

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Fiume	E	1	R

Costo dell'operazione 3 senza ridondanza:

(assumiamo che in media un fiume attraversi tre nazioni)

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Fiume	E	1	R
Nazione	E	1	R
Attraversa	R	3	R

Costo dell'operazione 4 con ridondanza:

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Lago	E	1	R

Costo dell'operazione 4 senza ridondanza:
(assumiamo che in media un lago bagni due nazioni)

Concetto	Tipo Concetto	Accessi	Tipo Accesso
Lago	E	1	R
Nazione	E	2	R
Bagna	R	2	R

Decisione sulle ridondanze

Analizzando la tabella dei volumi delle entità e delle operazioni abbiamo che il costo totale delle operazioni annualmente è:

Operazione	Numero R	Numero W	Totale
Op. 1 ridondante	1	2	150
Op. 1 non ridondante	2	1	120
Op. 2 ridondante	1	2	100
Op. 2 non ridondante	2	1	80
Op. 3 ridondante	1	0	30
Op. 3 non ridondante	5	0	150
Op. 4 ridondante	1	0	40
Op. 4 non ridondante	5	0	200

Il costo totale di tutte le operazioni precedentemente analizzate viene sommato per ogni tipo di attributo derivato coinvolto:

Attributo	Costo totale
Lunghezza totale ridondante	130
Lunghezza totale non ridondante	230
Superficie totale ridondante	190
Superficie totale non ridondante	320

Risulta evidente il costo annuo maggiore per le operazioni non ridondanti, quindi la decisione per lo schema logico sarà quella di implementare la versione ridondante.

4.1.2 Rimozione delle generalizzazioni

Nell'entità Fiume rimuoviamo la generalizzazione introducendo un'entità Corso_D'acqua mantenendo gli attributi inizialmente definiti e aggiungendo un attributo Tipo a rappresentare se il corso d'acqua sia un semplice fiume, un emissario, un immissario o entrambi. Di conseguenza subiranno una variazione anche le relazioni Esce ed Entra che avranno cardinalità (0-1) rispetto a Corso_D'acqua.

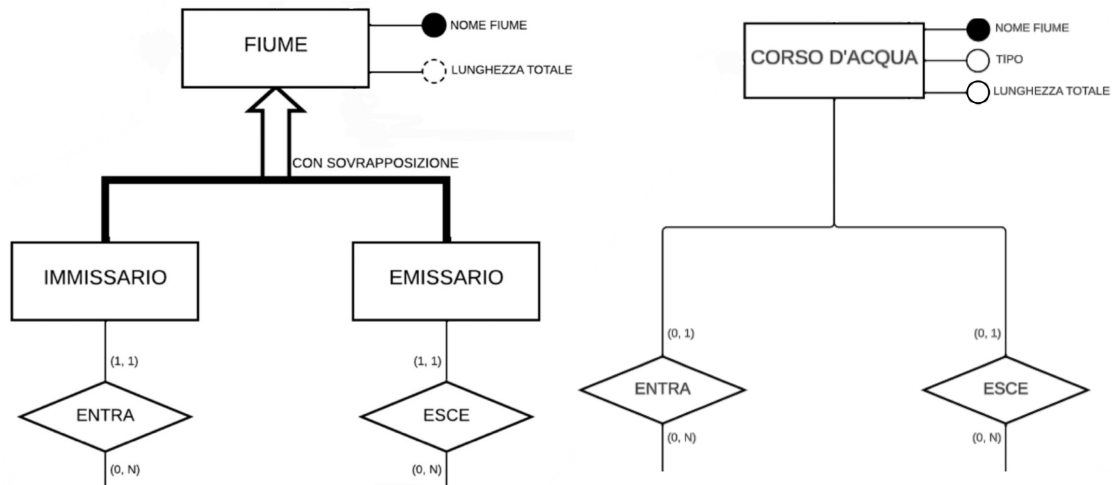


Figure 12: Rimozione della generalizzazione in Fiume

4.1.3 Rimozione degli attributi composti

Nell'entità Lago possiamo rimuovere l'attributo composto separando i due attributi che lo compongono come due attributi distinti: profondità e superficie_totale.

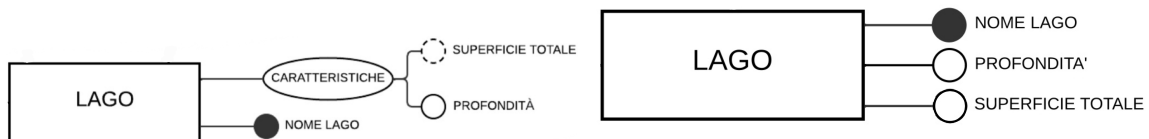


Figure 13: Rimozione dell'attributo composto in Lago

4.1.4 Definizione delle chiavi primarie

Come chiavi primarie andiamo ad utilizzare quelle individuate nello schema ER e nel caso di nazione andremo a scegliere Nome_Nazione come chiave primaria.

4.1.5 Schema ER ristrutturato

Poste le modifiche dei capitoli precedenti otteniamo il seguente schema ER.

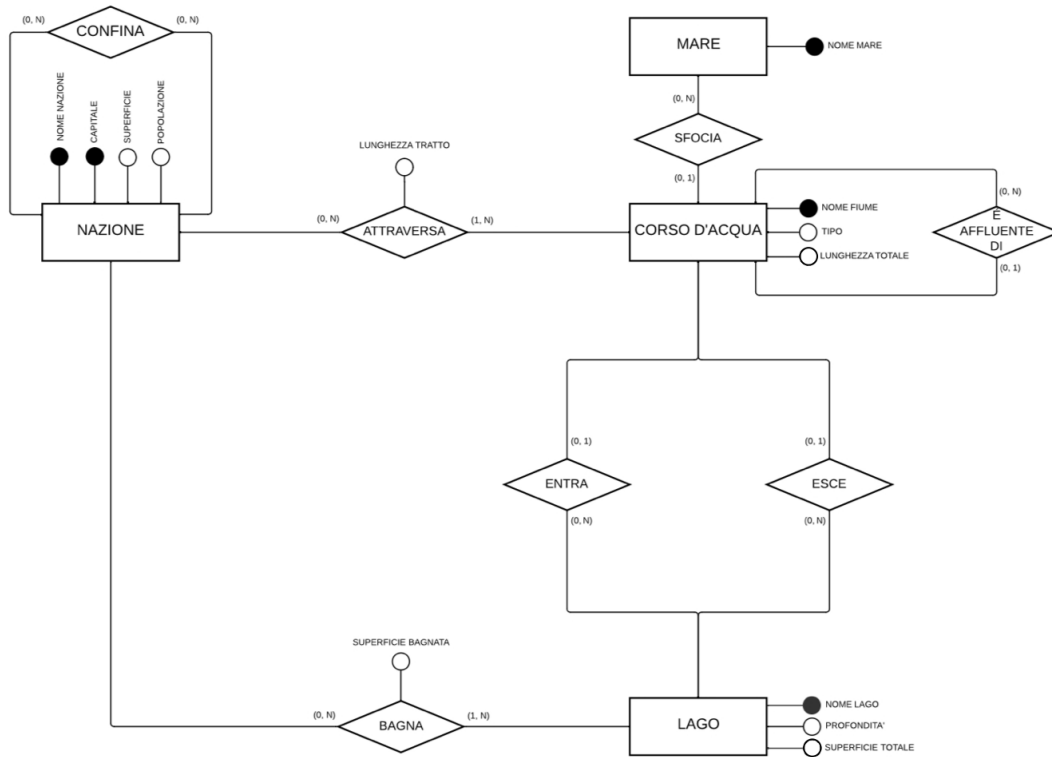
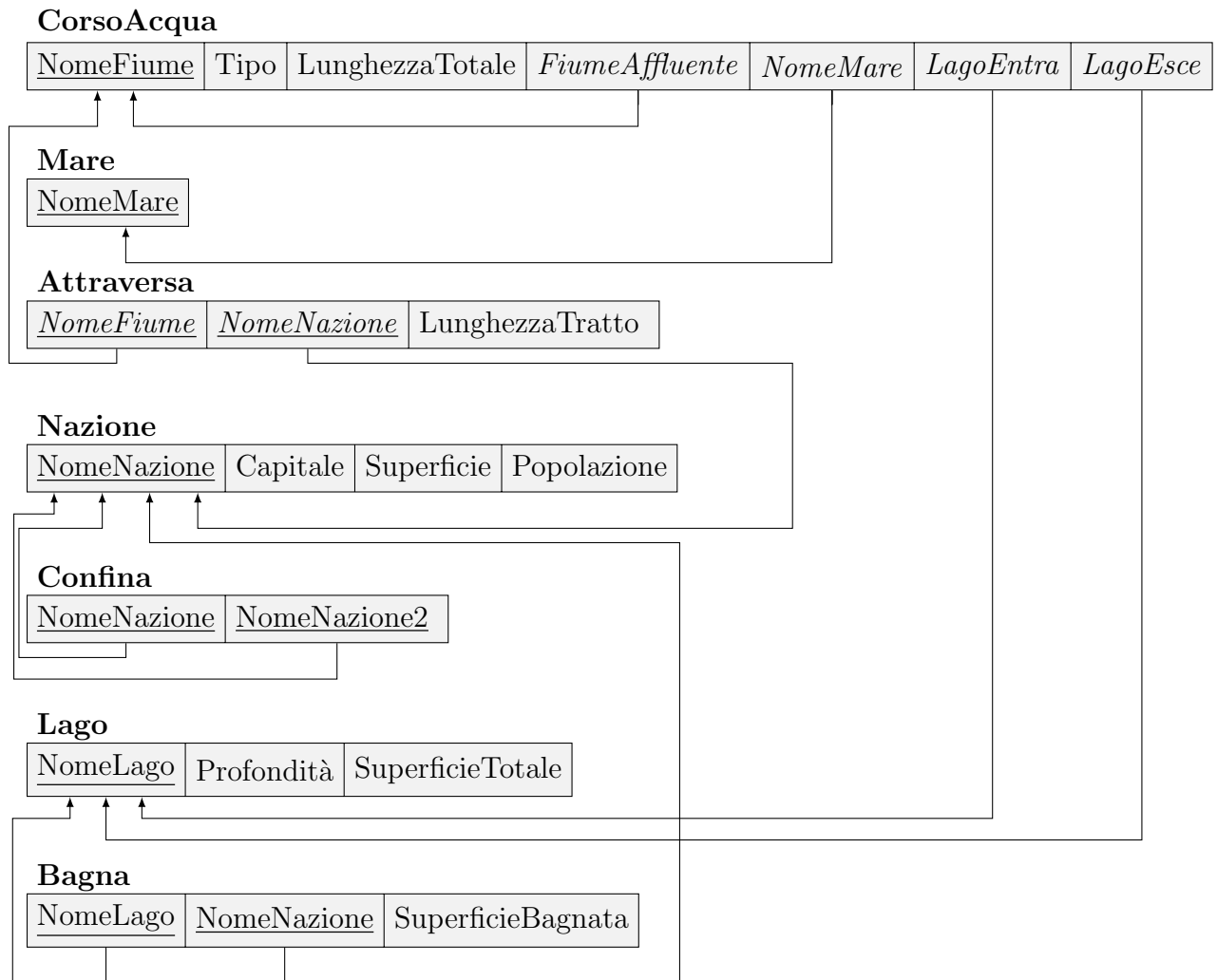


Figure 14: Schema ER ristrutturato

4.2 Conversione dello schema

Lo schema ER ristrutturato viene ora convertito in schema logico.

Nei vari schemi identificheremo le chiavi primarie come "Chiave Primaria" e le chiavi esterne come "*Chiave Esterna*".



4.2.1 Vincoli di integrità

- Capitale deve essere NOT NULL e UNIQUE per garantire le sue caratteristiche di chiave.
- Se viene aggiunto un corso d'acqua, deve attraversare almeno una nazione.
- Se viene aggiunto un lago, deve bagnare almeno una nazione.

4.3 Schema Logico

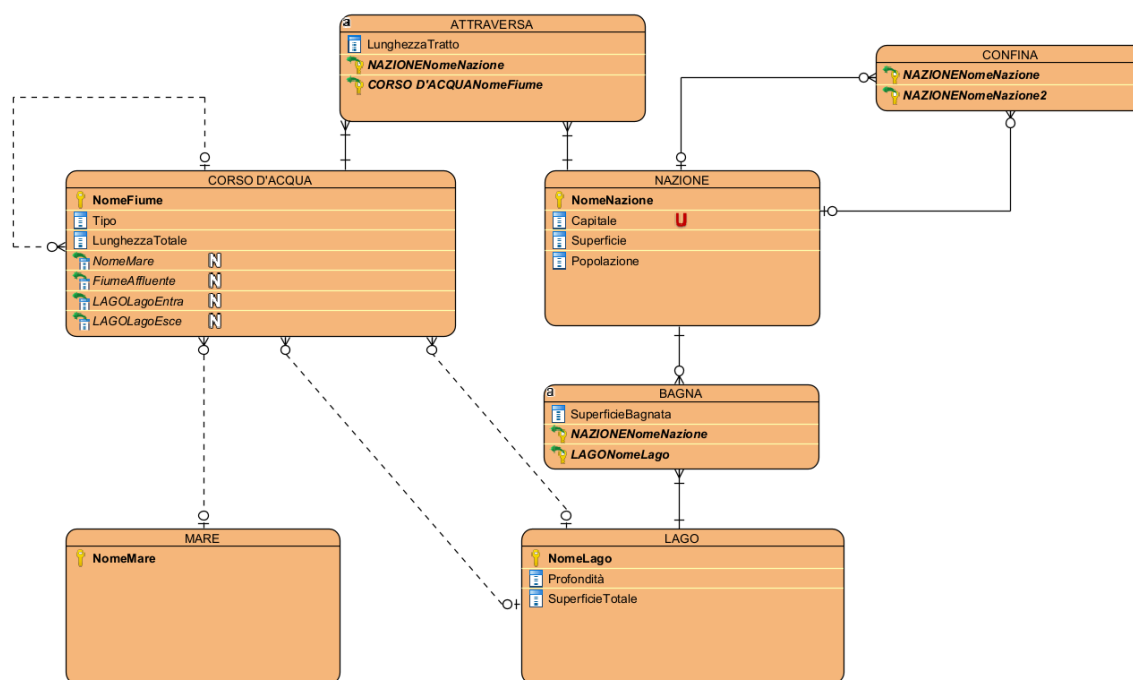


Figure 15: Schema logico del database

5 Progettazione Fisica

5.1 Scelta dei tipi

Viene ora definito un tipo per ogni attributo:

CorsoAcqua

Attributo	Tipo	Motivazione
NomeFiumne	varchar(60)	Si ritiene che 60 caratteri bastino per la definizione del nome di un fiume
Tipo	tipoFiume	Viene usata un'enumerazione con possibili valori Fiume, Emissario, Immissario, Emissario ed Immissario
LunghezzaTotale	smallint	Si ritiene che serva uno smallint per una rappresentazione corretta della lunghezza di un fiume in km

Mare

Attributo	Tipo	Motivazione
NomeMare	varchar(30)	Si ritiene che 30 caratteri bastino per rappresentare il nome dei mari europei

Attraversa

Attributo	Tipo	Motivazione
LunghezzaTratto	smallint	Si ritiene che serva uno smallint per una rappresentazione corretta della lunghezza di un fiume in km

Bagna

Attributo	Tipo	Motivazione
SuperficieBagnata	smallint	Si ritiene che serva uno smallint per una rappresentazione corretta della superficie bagnata da un lago in una nazione in km

Nazione

Attributo	Tipo	Motivazione
NomeNazione	varchar(20)	Si ritiene che 20 caratteri bastino per la rappresentazione dei nomi delle nazioni europee
Capitale	varchar(20)	Si ritiene che 20 caratteri bastino per la rappresentazione delle capitali europee
Superficie	integer	Si ritiene che serva un integer per una rappresentazione corretta della superficie di una nazione in km
Popolazione	real	Si ritiene che basti un real per la rappresentazione della popolazione delle nazioni europee

Lago

Attributo	Tipo	Motivazione
NomeLago	varchar(20)	Si ritiene che 20 caratteri bastino per la definizione del nome di un lago
Profondità	smallint	Si ritiene che uno smallint basti per la rappresentazione della profondità di un lago in km
SuperficieTotale	smallint	Si ritiene che serva uno smallint per una rappresentazione corretta della superficie di un lago in km

Tutti i dati decisi in questo capitolo relativi alle lunghezze sono basati sulla ricerca reale delle misure effettive europee.

5.2 Schema Fisico

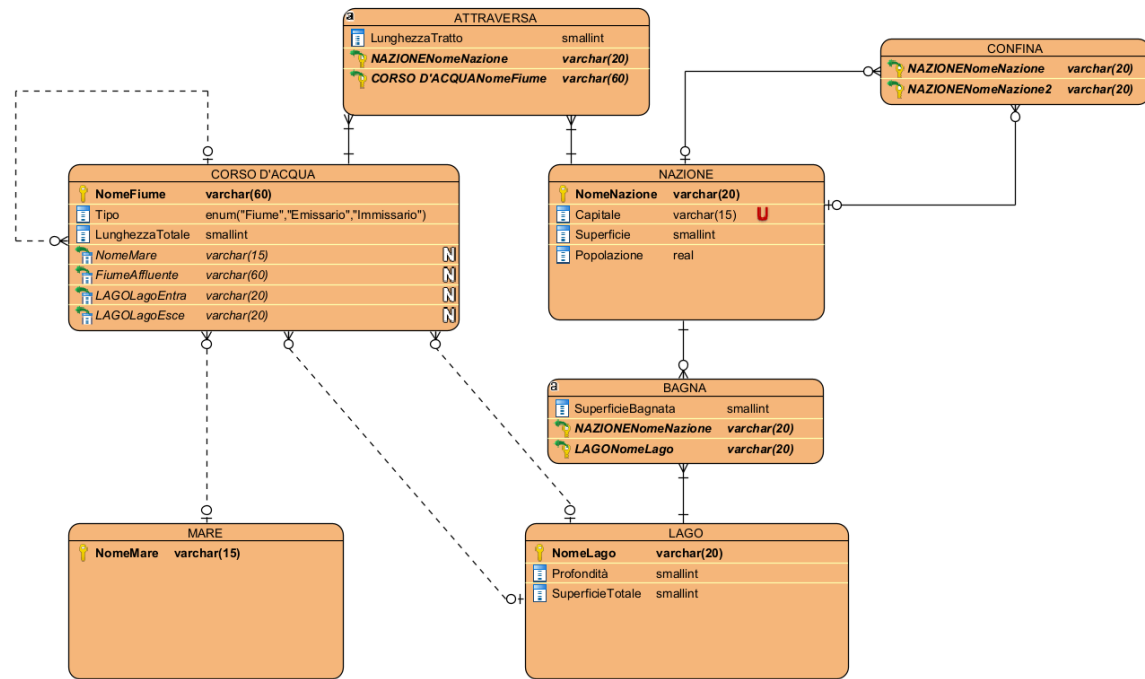


Figure 16: Schema fisico del database

6 Implementazione

6.1 Creazione del database e delle tabelle

Per la creazione delle tabelle è stato usato VisualStudioCode con PostgreSQL.

Creazione del DataBase

```
CREATE DATABASE "Laghi e Fiumi europei"
  WITH OWNER = postgres
  ENCODING = 'UTF8';
```

Creazione tabella Corso D'acqua

```
CREATE TYPE public.tipofiume AS ENUM
  ('Fiume', 'Emissario', 'Immissario', 'Emissario ed Immissario');

CREATE TABLE IF NOT EXISTS public."CORSO D\ ACQUA" (
  nomefiume character varying(60) COLLATE pg_catalog."default" NOT
    NULL,
  tipo tipofiume NOT NULL,
  lunghezzatotale integer NOT NULL,
  nomemare character varying(15) COLLATE pg_catalog."default",
  fiumeaffluente character varying(60) COLLATE pg_catalog."default"
    ,
  lagoentra character varying(20) COLLATE pg_catalog."default",
  lagoesce character varying(20) COLLATE pg_catalog."default",
  CONSTRAINT "CORSO D\ ACQUA_pkey" PRIMARY KEY (nomefiume),
  CONSTRAINT "CORSO D\ ACQUA_fiumeaffluente_fkey" FOREIGN KEY (
    fiumeaffluente)
    REFERENCES public."CORSO D\ ACQUA" (nomefiume) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,
  CONSTRAINT "CORSO D\ ACQUA_lagoentra_fkey" FOREIGN KEY (
    lagoentra)
    REFERENCES public.lago (nomelago) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,
  CONSTRAINT "CORSO D\ ACQUA_lagoesce_fkey" FOREIGN KEY (lagoesce
    )
    REFERENCES public.lago (nomelago) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,
  CONSTRAINT "CORSO D\ ACQUA_nomemare_fkey" FOREIGN KEY (nomemare
    )
    REFERENCES public.mare (nomemare) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE NO ACTION )
```

Creazione tabella Lago

```
CREATE TABLE IF NOT EXISTS public.lago (
    nomelago character varying(20) COLLATE pg_catalog."default" NOT
    NULL,
    "profondita" smallint NOT NULL,
    superficietotale smallint NOT NULL,
    CONSTRAINT lago_pkey PRIMARY KEY (nomelago) )
```

Creazione tabella Nazione

```
CREATE TABLE IF NOT EXISTS public.nazione (
    nomenazione character varying(20) COLLATE pg_catalog."default"
    NOT NULL,
    capitale character varying(20) COLLATE pg_catalog."default" NOT
    NULL,
    superficie integer NOT NULL,
    popolazione real NOT NULL,
    CONSTRAINT nazione_pkey PRIMARY KEY (nomenazione),
    CONSTRAINT nazione_capitale_key UNIQUE (capitale) )
```

Creazione tabella Mare

```
CREATE TABLE IF NOT EXISTS public.mare(
    nomemare character varying(30) COLLATE pg_catalog."default" NOT
    NULL,
    CONSTRAINT mare_pkey PRIMARY KEY (nomemare) )
```

Creazione tabella Attraversa

```
CREATE TABLE IF NOT EXISTS public.attraversa (
    lunghezzatratto smallint NOT NULL,
    nomenazione character varying(20) COLLATE pg_catalog."default"
    NOT NULL,
    nomefiume character varying(60) COLLATE pg_catalog."default" NOT
    NULL,
    CONSTRAINT attraversa_pkey PRIMARY KEY (nomefiume, nomenazione),
    CONSTRAINT attraversa_nomefiume_fkey FOREIGN KEY (nomefiume)
    REFERENCES public."CORSO D\’ ACQUA" (nomefiume) MATCH SIMPLE
    ON UPDATE NO ACTION
    ON DELETE NO ACTION,
    CONSTRAINT attraversa_nomenazione_fkey FOREIGN KEY (nomenazione)
    REFERENCES public.nazione (nomenazione) MATCH SIMPLE
    ON UPDATE CASCADE
    ON DELETE NO ACTION)
```

Creazione tabella Confina

```
CREATE TABLE IF NOT EXISTS public.confina (  
    nomenclazione character varying(20) COLLATE pg_catalog."default"  
        NOT NULL,  
    nomenclazione2 character varying(20) COLLATE pg_catalog."default"  
        NOT NULL,  
    CONSTRAINT confina_pkey PRIMARY KEY (nomenclazione, nomenclazione2),  
    CONSTRAINT confina_nomenclazione2_fkey FOREIGN KEY (nomenclazione2)  
        REFERENCES public.nazione (nomenclazione) MATCH SIMPLE  
        ON UPDATE CASCADE  
        ON DELETE NO ACTION  
        DEFERRABLE INITIALLY DEFERRED,  
    CONSTRAINT confina_nomenclazione_fkey FOREIGN KEY (nomenclazione)  
        REFERENCES public.nazione (nomenclazione) MATCH SIMPLE  
        ON UPDATE NO ACTION  
        ON DELETE NO ACTION  
        DEFERRABLE INITIALLY DEFERRED )
```

Creazione tabella Bagna

```
CREATE TABLE IF NOT EXISTS public.bagna (  
    nomelago character varying(20) COLLATE pg_catalog."default" NOT  
        NULL,  
    nomenclazione character varying(20) COLLATE pg_catalog."default"  
        NOT NULL,  
    superficiebagnata smallint NOT NULL,  
    CONSTRAINT bagna_pkey PRIMARY KEY (nomelago, nomenclazione),  
    CONSTRAINT bagna_nomelago_fkey FOREIGN KEY (nomelago)  
        REFERENCES public.lago (nomelago) MATCH SIMPLE  
        ON UPDATE NO ACTION  
        ON DELETE NO ACTION,  
    CONSTRAINT bagna_nomenclazione_fkey FOREIGN KEY (nomenclazione)  
        REFERENCES public.nazione (nomenclazione) MATCH SIMPLE  
        ON UPDATE CASCADE  
        ON DELETE NO ACTION )
```


6.2 Trigger

Trigger per la verifica dei vincoli d'integrità:

Se viene aggiunto un corso d'acqua, deve attraversare almeno una nazione.

```
CREATE OR REPLACE FUNCTION public.controlla_attraversamento()
    RETURNS trigger
    LANGUAGE 'plpgsql'
AS $BODY$
declare
new_NomeFiume varchar (60);
begin
    SELECT NomeFiume into new_NomeFiume from ATTRAVERSA
    where NomeFiume = new. NomeFiume ;
    IF not found
        THEN
            raise exception 'Il fiume non attraversa nessuna nazione';
            return null ;
        end IF;
end;
$BODY$;

CREATE OR REPLACE TRIGGER controlla_corso
    BEFORE INSERT
    ON public."CORSO D ACQUA"
    FOR EACH ROW
    EXECUTE FUNCTION public.controlla_attraversamento();
```

Se viene aggiunto un lago, deve bagnare almeno una nazione.

```
CREATE OR REPLACE FUNCTION public.controlla_bagna()
    RETURNS trigger
    LANGUAGE 'plpgsql'
AS $BODY$
declare
new_NomeLago varchar (20);
begin
    SELECT NomeLago into new_NomeLago from BAGNA
    where NomeLago = new.Nomelago ;
    IF not found
        THEN
            raise exception 'Il lago non bagna nessuna nazione';
            end IF;
end;
$BODY$;

CREATE OR REPLACE TRIGGER controlla_lago
    BEFORE INSERT
    ON public.lago
    FOR EACH ROW
    EXECUTE FUNCTION public.controlla_bagna();
```

6.3 Implementazione delle operazioni

1) Trovare tutte le nazioni bagnate da 3 o più laghi

```
SELECT NomeNazione
FROM BAGNA
GROUP BY NomeNazione
HAVING COUNT(DISTINCT NomeLago) >= 3;
```

2) Trovare tutte le nazioni bagnate da 1 o più laghi da cui esce un fiume che sfocia nel Mar Mediterraneo

```
SELECT NomeNazione
FROM BAGNA AS B1
WHERE EXISTS (SELECT *
              FROM BAGNA
              WHERE B1.NomeNazione = NomeNazione AND
              EXISTS (SELECT *
                    FROM "CORSO D'ACQUA"
                    WHERE LagoEsce = B1.NomeLago AND
                    NomeMare = "Mar Baltico"));
```

3) Trovare il numero di fiumi che sono immissari in un lago che ha al più 3 emissari

```
SELECT COUNT(*)
FROM "CORSO D'ACQUA" AS C1
WHERE EXISTS (SELECT *
              FROM "CORSO D'ACQUA"
              WHERE C1.LagoEntra=LagoEsce
              GROUP BY LagoEsce
              HAVING COUNT (DISTINCT *) < 4);
```

4) Trovare tutti i fiumi che sono sia immissari che emissari

```
SELECT NomeFiume
FROM "CORSO D'ACQUA"
WHERE LagoEntra AND LagoEsce NOT NULL;
```

5) Aggiornamento della superficie di un lago

```
UPDATE LAGO
SET SuperficieTotale = 146
WHERE NomeLago="Lago di Como";

UPDATE BAGNA
SET SuperficieBagnata = SuperficieBagnata + 1
WHERE NomeLago = "Lago di Como" AND NomeNazione = "Italia";
```

6) Aggiornamento della lunghezza di un fiume

```
UPDATE "CORSO D'ACQUA"
SET LunghezzaTotale = 652,8
WHERE NomeFiume = "Po";

UPDATE ATTRAVERSA
SET LunghezzaTratto = LunghezzaTratto + 1
WHERE NomeFiume = "Po" AND NomeNazione = "Italia";
```

7) Calcolare la lunghezza totale di un fiume

```
SELECT LunghezzaTotale
FROM "CORSO D'ACQUA"
WHERE NomeFiume = "Po";
```

8) Calcolare la superficie totale di un lago

```
SELECT SuperficieTotale
FROM LAGO
WHERE NomeLago = "Lago di Como";
```

9) Calcolare la distribuzione dei fiumi nelle nazioni

```
SELECT NomeNazione, COUNT(NomeFiume)
FROM Attraversa
GROUP BY NomeNazione;
```

10) Calcolare la distribuzione dei laghi nelle nazioni

```
SELECT NomeNazione, COUNT(NomeLago)
FROM Bagna
GROUP BY NomeNazione;
```

7 Analisi dei dati in R

7.1 Popolamento della base di dati

Per popolare la base di dati abbiamo usato R Studio. Viene riportato tutto il codice di seguito.

Pacchetti usati

```
library("RPostgreSQL")
library("DBI")
library("odbc")
library("dplyr")
library("readr")
library("tidyverse")
library("readxl")
```

Connessione alla base di dati

```
con <- dbConnect(RPostgres::Postgres(),
                 dbname="postgres",
                 host="localhost",
                 port=5432,
                 user="postgres",
                 password="password")
```

Import da excel in R

Le tabelle sono state da noi costruite al fine di un popolamento ottimale della base di dati.

```
CorsoDAcqua<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Corso D'Acqua")
Lago<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Lago")
Nazione<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Nazione")
Mare<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Mare")
Attraversa<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Attraversa")
Confina<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Confina")
Bagna<-read_excel
  ("Contenuto tabelle.xlsx",sheet="Bagna")
```

Popolamento della base di dati

È stato necessario costruire una transazione con controlli differiti in modo da non far scattare alcun trigger o violare alcun vincolo di integrità durante il popolamento.

```
dbExecute(con, "BEGIN;")
dbExecute(con, "SET session_replication_role = 'replica';")
tryCatch({

dbWriteTable
  ( con,"CORSO D ACQUA", CorsoDAcqua , append = TRUE, row.
    names = FALSE)
dbWriteTable
  ( con,"attraversa", Attraversa , append = TRUE, row.
    names = FALSE)
dbWriteTable
  ( con,"bagna", Bagna , append = TRUE, row.names = FALSE)
dbWriteTable
  ( con,"confina", Confina , append = TRUE, row.names =
    FALSE)
dbWriteTable
  ( con,"lago", Lago , append = TRUE, row.names = FALSE)
dbWriteTable
  ( con,"mare", Mare , append = TRUE, row.names = FALSE)
dbWriteTable
  ( con,"nazione", Nazione , append = TRUE, row.names =
    FALSE)

dbExecute(con, "SET session_replication_role = 'origin';")
dbExecute(con, "COMMIT;")
},error = function(e) {
  dbExecute(con, "SET session_replication_role = 'origin';")
  dbExecute(con, "ROLLBACK;")
  print(paste("Errore durante l'inserimento:", e$message))
})
```

7.2 Analisi grafica delle operazioni

Operazione 9

```
Operazione9 <-dbGetQuery ( con,  
                           "SELECT nomenclazione, COUNT(nomefiume)  
                           FROM   attraversa  
                           GROUP BY nomenclazione;")  
  
ylim<-c(0,10)  
  
barplot (as.numeric ( Operazione9$count),  
         main = " Distribuzione dei fiumi ",  
         ylab = " Numero ",  
         names.arg = Operazione9$nomenclazione ,  
         col = "blue",  
         las=2,  
         ylim=ylim  
)
```

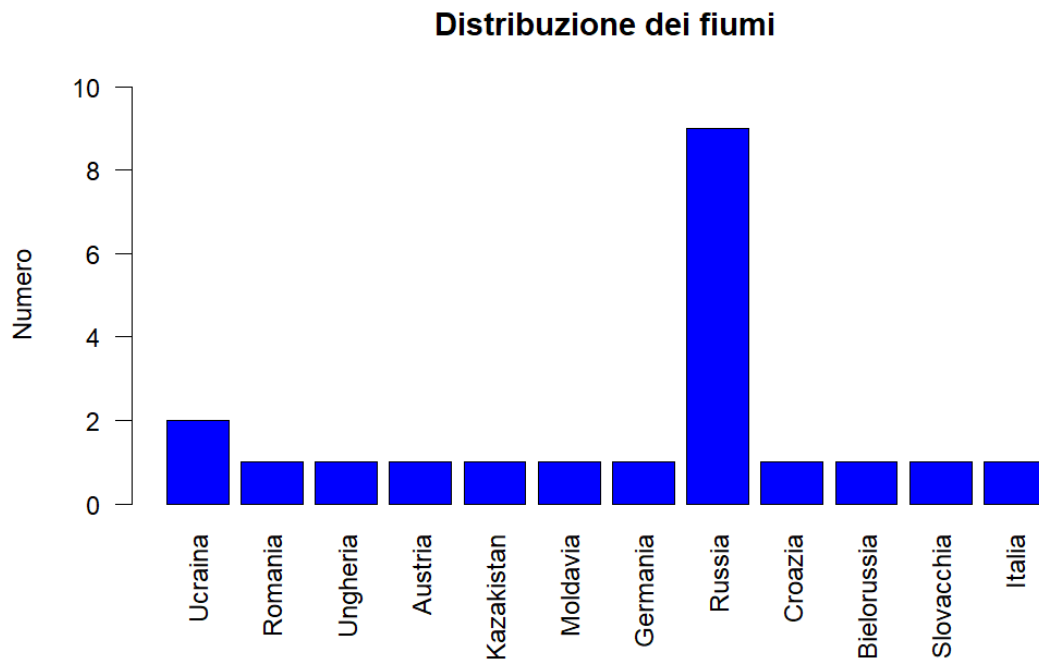


Figure 17: Distribuzione dei fiumi nelle nazioni europee

Operazione 9.1

Abbiamo trovato interessante implementare una seconda operazione collegata alla precedente che calcola la distribuzione dei fiumi nelle nazioni, normalizzata per la superficie delle nazioni stesse.

```
Operazione9.1 <- dbGetQuery (con,
  "SELECT n.nomenclazione, COUNT(a.nomefiume) as numero_fiumi,
    n.superficie
  FROM attraversa a
  JOIN nazione n ON a.nomenclazione = n.nomenclazione
  GROUP BY n.nomenclazione, n.superficie;")

Operazione9.1$numero_normalizzato <- Operazione9.1$numero_fiumi /
  Operazione9.1$superficie

ylim <- c(0, 30)

barplot (Operazione9.1$numero_normalizzato* 1e6,
  main = "Distribuzione normalizzata dei fiumi per nazione",
  ylab = "Numero normalizzato",
  names.arg = Operazione9.1$nomenclazione,
  col = "blue",
  las = 2,
  ylim=ylim
)
```

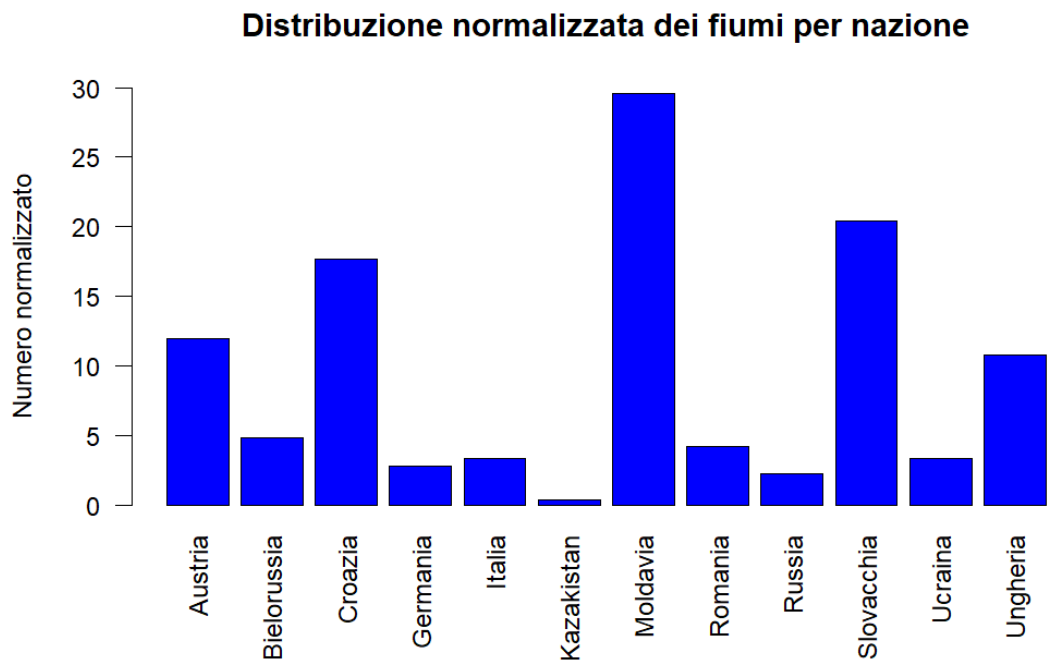


Figure 18: Distribuzione dei normalizzata fiumi nelle nazioni europee

Operazione 10

```
Operazione10 <-dbGetQuery (con,  
                           "SELECT nomenclazione ,  
                             COUNT(nomelago)  
                             FROM   bagna  
                             GROUP BY nomenclazione;")  
  
barplot (as.numeric (Operazione10$count),  
        main = " Distribuzione dei laghi ",  
        ylab = " Numero ",  
        names.arg = Operazione10$nomenclazione ,  
        col = "lightblue",  
        las=2  
)
```

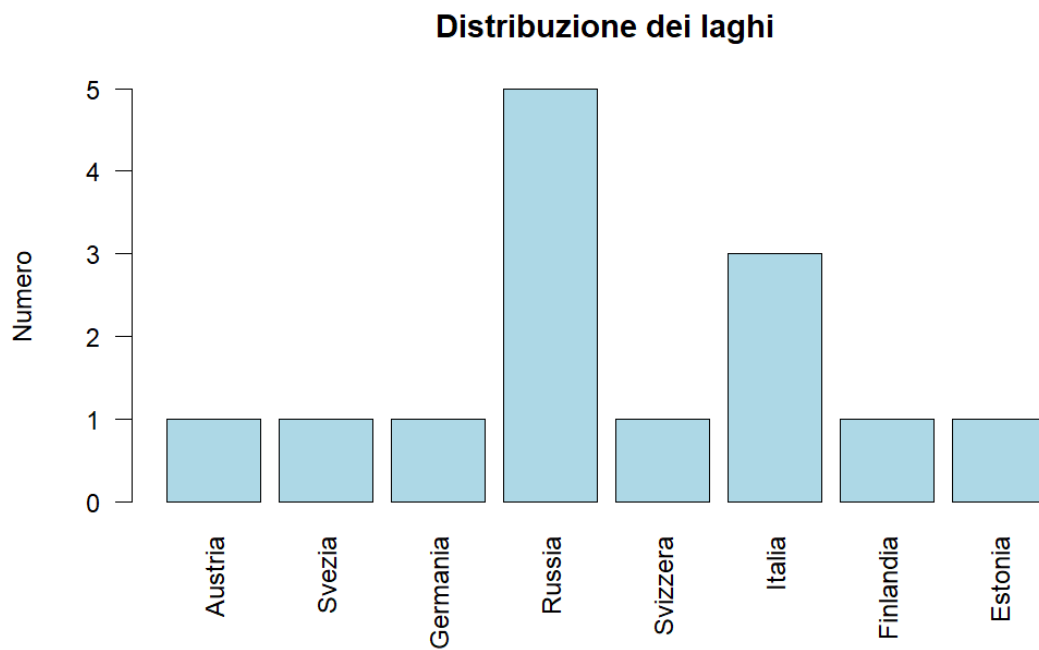


Figure 19: Distribuzione dei laghi nelle nazioni europee

Operazione 10.1

Implementazione simile alle operazioni precedenti, andiamo a calcolare la distribuzione dei laghi nelle nazioni, normalizzata per la superficie delle nazioni.

```
Operazione10.1 <- dbGetQuery(con,
  "SELECT n.nomenclazione, COUNT(b.nomenclago) as numero_laghi,
    n.superficie
  FROM bagna b
  JOIN nazione n ON b.nomenclazione = n.nomenclazione
  GROUP BY n.nomenclazione, n.superficie;")

Operazione10.1$numero_normalizzato <- Operazione10.1$numero_laghi /
  Operazione10.1$superficie

ylim <- c(0, 25)

barplot(Operazione10.1$numero_normalizzato* 1e6,
  main = "Distribuzione normalizzata dei laghi per nazione",
  ylab = "Numero normalizzato",
  names.arg = Operazione10.1$nomenclazione,
  col = "lightblue",
  las = 2,
  ylim=ylim
)
```

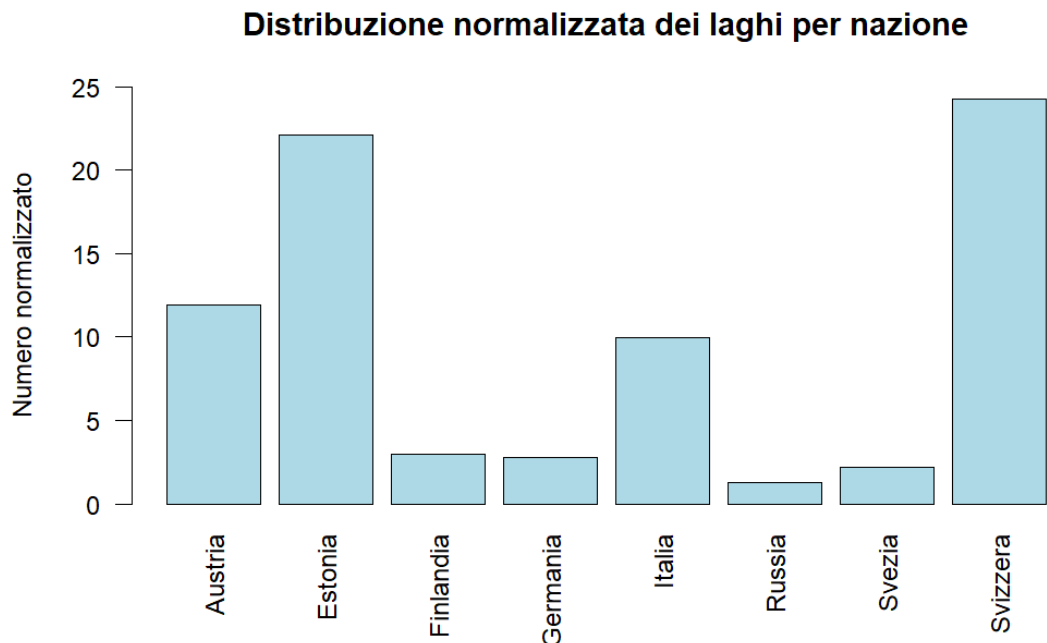
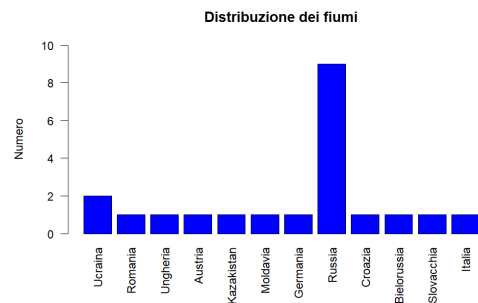


Figure 20: Distribuzione normalizzata dei laghi nelle nazioni europee

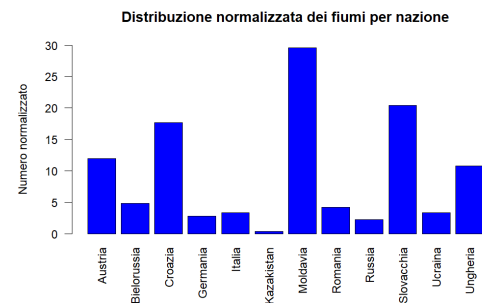
7.2.1 Considerazioni grafiche

Da come evince nel confronto tra la figura (a) e la figura (b) possiamo notare che la Russia sia lo stato con la distribuzione dei fiumi più alta. Invece, considerando un valore normalizzato per la superficie dello stato possiamo notare come la Moldavia spicchi come maggioritaria.

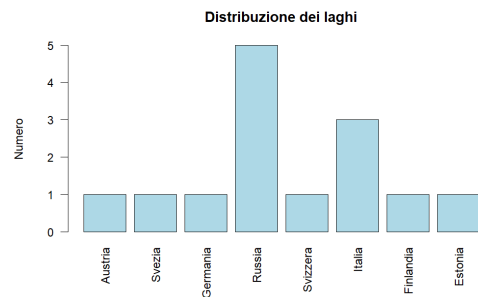
Facciamo lo stesso confronto anche tra la figura (c) e la figura (d), dove la Russia appaia come lo stato con la maggiore distribuzione dei laghi. Invece, rapportandoci a dei dati normalizzati per la superficie degli stati notiamo come la Svizzera spicchi come maggioritaria.



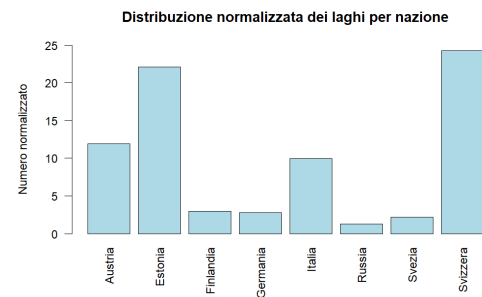
(a) Fiumi



(b) Fiumi normalizzati



(c) Laghi



(d) Laghi normalizzati

Da questa analisi possiamo concludere, affidandoci ai dati normalizzati in quanto più veritieri e rappresentativi, che la Moldavia sia lo stato europeo con la distribuzione dei fiumi più alta e che la Svizzera sia la nazione europea con la distribuzione dei laghi più alta.

7.3 Stima della quantità di memoria

Per stimare la quantità di memoria occupata da ogni singola tabella e di conseguenza anche dall'intera base di dati prendiamo in considerazione i seguenti valori:

- I valori numerici occupano dai 2 agli 8 Byte a seconda del tipo.
- I valori varchar occupano $8N$ Byte dove N è il numero di caratteri utilizzati.

Per ogni colonna della tabella andremo a moltiplicare il numero di valori per i Byte richiesti dal tipo di attribuito utilizzato e infine, sommeremo questi valori per ottenere la quantità di memoria occupata dalla tabella.

In seguito, una volta eseguiti questi passaggi per ogni tabella della base di dati, andremo a sommare tutti i valori ottenuti per avere una stima della quantità di memoria occupata dalla base di dati.

Tabella Corso D'Acqua

$$(12 \cdot 8 \cdot 60) \cdot 2 + (12 \cdot 8 \cdot 23) + (12 \cdot 8) + (12 \cdot 8 \cdot 15) + (12 \cdot 8 \cdot 20) \cdot 2 \\ = 19104 \text{ Byte} \approx 19 \text{ KByte}$$

Tabella Attraversa

$$(21 \cdot 2) + (21 \cdot 8 \cdot 20) + (21 \cdot 8 \cdot 60) = 13482 \text{ Byte} \approx 13,5 \text{ KByte}$$

Tabella Bagna

$$(14 \cdot 8 \cdot 20) + (14 \cdot 8 \cdot 20) + (14 \cdot 2) = 4508 \text{ Byte} \approx 4,5 \text{ KByte}$$

Tabella Confina

$$(201 \cdot 8 \cdot 20) + (201 \cdot 8 \cdot 20) = 64320 \text{ Byte} \approx 64,5 \text{ KByte}$$

Tabella Lago

$$(11 \cdot 8 \cdot 20) + (11 \cdot 2) + (1 \cdot 2) = 1804 \text{ Byte} \approx 1,8 \text{ KByte}$$

Tabella Mare

$$(12 \cdot 8 \cdot 30) = 2880 \text{ Byte} \approx 2,9 \text{ KByte}$$

Tabella Nazione

$$(51 \cdot 8 \cdot 20) + (51 \cdot 8 \cdot 20) + (51 \cdot 8) + (51 \cdot 8) = 17136 \text{ Byte} \approx 17,1 \text{ KByte}$$

Spazio totale delle tabelle

In totale le tabelle utilizzate per la base di dati occupano approssimativamente

$$123234 \text{ Byte} = 123,2 \text{ KByte}$$